

I. INDICE

II. INTRODUCCIÓN.....	3
III. OBJETIVOS.....	5
IV. CLASES ASOCIADAS A LAS OPERACIONES DE GESTIÓN DE FICHEROS Y DIRECTORIOS: CREACIÓN, BORRADO, COPIA, MOVIMIENTO, ENTRE OTRAS.....	5
4.1. Flujos o Streams. Tipos	8
4.1.1. Flujos de bytes (Byte streams).....	9
4.1.2. Flujos de caracteres (Character streams).....	12
V. FORMAS DE ACCESO A UN FICHERO	14
5.1 Operaciones sobre ficheros.....	14
5.1.1 Operaciones sobre ficheros secuenciales.....	14
5.1.2 Operaciones sobre ficheros aleatorios.....	15
VI. CLASES PARA GESTIÓN DE FLUJOS DE DATOS DESDE/HACIA FICHEROS	16
6.1 Ficheros de texto	16
6.2 Ficheros binarios.....	21
6.3 Objetos en ficheros binarios.....	25
6.3.1. Problemas con los ficheros de objetos	28
6.4 Ficheros de acceso aleatorio	29
VII. TRABAJO CON FICHEROS XML: ANALIZADORES SINTÁCTICOS (PARSER) Y VINCULACIÓN (BINDING).....	34
7.1 Acceso a ficheros XML con DOM	35
7.2 Acceso a ficheros XML con SAX	40
7.3 Serialización de objetos a XML	42
7.4 Conversión de ficheros XML a otro formato	45
VIII. EXCEPCIONES: DETECCIÓN Y TRATAMIENTO	48

IX. EJERCICIO OBLIGATORIO	51
X. TEST DE AUTOEVALUACIÓN.....	52
XI. GLOSARIO.....	55
XII. RESUMEN	56
XIII. BIBLIOGRAFÍA.....	57
XIV. LECTURAS RECOMENDADAS PARA LA UNIDAD	57

II. INTRODUCCIÓN

A la hora de trabajar con datos, éstos solo permanecen en el ordenador mientras se está ejecutando un programa o proceso.

Cuando se sale del programa, esta información desaparece y se pierde la información, por lo que surge la necesidad de almacenarlos más allá del tiempo en el que se estén utilizando los programas. Es decir, al salir de un programa los datos generados se deben guardar en algún lugar que permita su recuperación desde el mismo u otros programas. Por tanto, hay que encontrar la manera de que estos datos sean persistentes.

Por lo tanto ¿Qué se puede hacer para que la información sea persistente y no se pierda? Y lo más importante ¿Cómo se puede recuperar? **La solución a todo esto se va a ver en esta unidad.**

Para almacenar la información, los sistemas operativos ofrecen una estructura de datos de alto nivel llamada fichero que distribuye los datos en dispositivos externos de almacenamiento. De esta forma, toda la información manejada por el sistema informático se almacena en ficheros.

En multitud de ocasiones **fichero** y **archivo** son tratados como sinónimos aunque algunos autores diferencian los archivos de los ficheros estableciendo que los archivos son ficheros que contienen datos. De forma que los ficheros podrán contener código o datos, haciendo el término más general ya que engloba a los archivos.

Un fichero o archivo es un conjunto de información (un conjunto de bits almacenados) relacionada de forma lógica a la que se da un nombre para identificarla de forma unívoca.

Los ficheros son un mecanismo abstracto que proporciona el Sistema Operativo para almacenar y consultar información, independientemente de la forma en que se almacene.

Un fichero también puede verse como un tipo abstracto de datos, es decir, un objeto que tiene asociado unos atributos y unas operaciones.

Los ficheros han sufrido un gran cambio desde sus inicios. En principio eran meros sistemas de almacenamiento de información sin estructura ninguna. Pasando posteriormente por su estructuración en campos y registros. Hasta por último estructurarse en complicadas bases de datos, que son una colección de ficheros relacionados.

Un fichero está formado por un conjunto de registros o líneas y cada registro por un conjunto de campos relacionados, por ejemplo, un fichero de empleados puede contener datos de los empleados de una empresa, un fichero de texto puede contener líneas de texto correspondientes a líneas impresas en una hoja de papel. La manera en

que se agurpan los datos en el fichero depende completamente de la persona que los esté diseñando.

En esta unidad el alumno va a dominar el uso básico de archivos en Java para conseguir persistencia de datos. Para ello serán presentados conceptos básicos sobre archivos y algunas de las clases de la biblioteca estándar de Java para su creación y manipulación. Además, el uso de esas bibliotecas permitirá introducir algunos conceptos “avanzados” de programación en Java: como las excepciones, para tratar posibles errores durante la ejecución de un programa, y manipulación de datos a bajo nivel, para transformar nuestros datos a vectores de bytes.

Además, se utilizarán diferentes librerías para convertir **ficheros XML** a otros formatos y así poder crear documentos HTML.

III. OBJETIVOS

- Utilizar clases para la gestión de ficheros y directorios.
- Valorar las ventajas y los inconvenientes de las distintas formas de acceso.
- Utilizar las operaciones básicas para acceder a ficheros de acceso secuencial y aleatorio.
- Utilizar clases para almacenar y recuperar información almacenada en un fichero XML.
- Utilizar clases para convertir a otro formato información contenida en un fichero XML.
- Gestionar excepciones.
- Serializar objetos Java a representaciones XML

IV. CLASES ASOCIADAS A LAS OPERACIONES DE GESTIÓN DE FICHEROS Y DIRECTORIOS: CREACIÓN, BORRADO, COPIA, MOVIMIENTO, ENTRE OTRAS

El paquete **java.io** contiene las clases para manejar la entrada/salida en Java, por lo que es importante que **importemos** dicho paquete cuando trabajemos con ficheros. Antes de ver las clases que leen y escriben datos en ficheros vamos a manejar la clase **File**. La clase **File** proporciona un conjunto de utilidades relacionadas con los ficheros que nos aportan información como pueden ser su nombre, atributos, directorios, etc. La clase **File** representa un fichero o directorio de nuestro sistema de archivos. También es útil ya que puede pasarse como parámetro para las principales clases que manejan ficheros, como **FileReader**, **FileInputStream**, **FileWriter**, etc. Algunos de sus constructores son:

- **File(String directorio):** *new File("/directorio/fichero.txt");* para Linux, *new File("C:\\directorio\\fichero.txt");* para Windows
- **File(String directorio, String nombrefichero):** *new File ("directorio", "fichero.txt");*
- **File(File directorio, String nombrefichero):** *new File (new File ("directorio"), "fichero.txt");*
- **File(Uri uri):** *new File(new URI(file:///c:/archivo.txt))*

A continuación un ejemplo donde se definen diversas formas para declarar un fichero en plataformas windows:

```
File fichero1 = new File("C:\\AccesoDatos\\UD1\\Ejercicios\\ejemplo1.txt");
//Crea una instancia de la clase File llamada fichero1 pasándole como
parámetro un String con la ruta absoluta del fichero fichero1.txt

String directorio = "C:/AccesoDatos/UD1/Ejercicios";
//Crea un String con la ruta absoluta del directorio Ejercicios

File fichero2 = new File(directorio, "ejemplo2.txt");
//Crea una instancia de la clase File llamada fichero2 pasándole como
parámetro la variable directorio de tipo String y otro String con el nombre
del fichero ejemplo2.txt y así crear el fichero
C:/AccesoDatos/UD1/Ejercicios/ejemplo2.txt

File direc = new File(directorio);
//Crea una instancia de la clase File llamada direc pasándole como parámetro
la variable directorio de tipo String creada anteriormente y así crear el
directorio C:/AccesoDatos/UD1/Ejercicios/

File fichero3 = new File(direc, "ejemplo3.txt");
//Crea una instancia de la clase File llamada fichero3 pasándole como
parámetro la variable direc de tipo String y otro String con el nombre del
fichero ejemplo3.txt y así crear el fichero
C:/AccesoDatos/UD1/Ejercicios/ejemplo3.txt

File fichero4 =
new File(new URI("file:///C:/AccesoDatos/UD1/Ejercicios/ejemplo4.txt"));
//Crea una instancia de la clase File llamada fichero4 pasándole como
parámetro una URI que se crea pasándole el nombre y ubicación del fichero a
crear C:/AccesoDatos/UD1/Ejercicios/ejemplo4.txt
```

Algunos de los métodos más importantes de la clase **FILE** son:

Método	Descripción	Parámetros	Dato devuelto
exists	Indica si existe o no el fichero.	Ninguno.	boolean
isDirectory	Indica si el objeto File es un directorio.	Ninguno.	boolean
isFile	Indica si el objeto File es un fichero.	Ninguno.	boolean
isHidden	Indica si el objeto File esta oculto.	Ninguno.	boolean
getAbsolutePath	Devuelve una cadena con la ruta absoluta del fichero o directorio.	Ninguno.	String

canRead	Indica si se puede leer.	Ninguno.	boolean
canWrite	Indica si se puede escribir.	Ninguno.	boolean
canExecute	Indica si se puede ejecutar.	Ninguno.	boolean
getName	Devuelve una cadena con el nombre del fichero o directorio.	Ninguno.	String
getParent	Devuelve una cadena con el directorio padre.	Ninguno.	String
listFiles	Devuelve un array de File con los directorios hijos. Solo funciona con directorios.	Ninguno.	Array de File
list	Devuelve un array de String con los directorios hijos. Solo funciona con directorios.	Ninguno.	Array de String
mkdir	Permite crear el directorio en la ruta indicada. Solo se creara si no existe.	Ninguno.	boolean
mkdirs	Permite crear el directorio en la ruta indicada, también crea los directorios intermedios. Solo se creara si no existe.	Ninguno.	boolean
createNewFile	Permite crear el fichero en la ruta indicada. Solo se creará si no existe. Debemos controlar la excepcion con IOException.	Ninguno.	boolean

Tabla 1.1. Métodos de la clase FILE

El siguiente ejemplo muestra la lista de ficheros en el directorio actual. Utiliza el método **list()**, que devuelve un array de Strings con los nombres de los ficheros.

```
import java.io.*;
public class VerDirectorios {
    public static void main(String[] args) {
        System.out.println("Archivos en el directorio actual:");
        File f = new File(".");
        String[] archivos = f.list();
        for (int i = 0; i < archivos.length; i++) {
            System.out.println(archivos[i]);
        }
    }
}
```

El siguiente ejemplo crea un directorio (de nombre *NUEVODIR*) en el directorio actual. A continuación, crea dos ficheros vacío en él.

```
import java.io.*;
public class CrearDirectorio {
    public static void main(String[] args) {
        File d = new File("NUEVODIR"); //directorio creado a partir del actual
        File f1 = new File(d,"FICHERO1.TXT");
        File f2 = new File(d,"FICHERO2.TXT");

        d.mkdir();//CREAR DIRECTORIO

        try {
            if (f1.createNewFile())
                System.out.println("FICHERO1 creado correctamente...");
            else
                System.out.println("No se ha podido crear FICHERO1...");
            if (f2.createNewFile())
                System.out.println("FICHERO2 creado correctamente...");
            else
                System.out.println("No se ha podido crear FICHERO2...");
        } catch (IOException ioe) {ioe.printStackTrace();}

        f1.renameTo(new File(d,"FICHERO1NUEVO"));//renombrar FICHERO1

        if(f1.delete()) //borrar FICHERO1
            System.out.println("Fichero 1 borrado...");
        else
            System.out.println("No se ha podido borrar el Fichero 1...");

        if(f2.delete())
            System.out.println("Fichero 2 borrado...");
        else
            System.out.println("No se ha podido borrar el Fichero 2...");

        if(d.delete()) //borrar DIRECTORIO
            System.out.println("Directorio NUEVODIR borrado...");
        else
            System.out.println("No se ha podido borrar el directorio
NUEVODIR...");

        try {
            File f3 = new File("NUEVODIR/FICHERO3.TXT");
            f3.createNewFile();
        } catch (IOException ioe) {ioe.printStackTrace();}
    }
}
```

El método ***delete()*** borra un fichero o un directorio. En el ejemplo anterior no se puede borrar el directorio creado porque contiene ficheros. Antes de eliminarlo, hay

que eliminar los ficheros que contenga. El método **createNewFile()** puede lanzar la excepción **IOException**, por eso se utiliza el bloque **try-catch**.

4.1. Flujos o Streams. Tipos

Todo dato tiene un origen de entrada o un destino de salida (convenientemente en inglés: I/O). Un programa trabaja con datos que provienen de una entrada la cual puede ser un teclado, un archivo en disco o un socket a través de la red. Así mismo, un programa genera resultados enviándolos a una salida la cual puede ser la pantalla, un archivo en disco o un socket a través de la red. Por lo tanto, hay distintos orígenes de entrada y destinos de salida para los datos. En Java, todos estos se consideran flujos (en inglés: **streams**) que podemos controlar mediante la API estándar del paquete **java.io**. Gracias a esto el programador siempre puede recurrir al mismo grupo de clases independientemente del caso a tratar.

Así que en Java se tiene un flujo de entrada (en inglés: **input stream**) por el cual se reciben los datos desde el exterior del programa, o un flujo de salida (en inglés: **output stream**) el cual envía los datos hacia un cierto destino. Se definen dos tipos de flujos:

- **Flujos de bytes (8-bit):** Realizan operaciones de entradas y salidas de bytes y su uso está orientado a la lectura/escritura de datos binarios. Todas las clases de flujos de bytes descienden de las clases **InputStream** y **OutputStream** de las cuales heredan otras subclases que implementan en formas más concretas la misma tarea, es decir, controlar las diferencias entre los distintos dispositivos de entrada/salida.
- **Flujos de caracteres (16-bit):** Es un tipo de flujo de caracteres en codificación Unicode, listo para la internacionalización, ideal para trabajar con texto plano. Las clases principales para manejar estos flujos son las clases abstractas **Reader** y **Writer** de las cuales heredan otras subclases que implementan en formas más concretas la misma tarea. Cualquiera de estas clases realiza la conversión correspondiente de byte a char para leer o de char a byte para escribir.

4.1.1. Flujos de bytes (Byte streams)

La clase **InputStream** representa las clases que producen entradas de distintas fuentes, estas fuentes puede ser: un array de bytes, un objeto String, un fichero, una "tubería" (un ejemplo las pipe de UNIX), una secuencia de otros flujos, otras fuentes como la conexión a Internet, etc. Los tipos de **InputStream** se resumen en la siguiente tabla:

CLASE	Función
ByteArrayInputStream	Permite usar un espacio de almacenamiento intermedio de memoria
StringBufferInputStream	Convierte un String en un InputStream
FileInputStream	Flujo de entrada hacia fichero, se usa para leer información de un fichero
PipedInputStream	Implementa el concepto de “tubería”
FilterInputStream	Proporciona funcionalidad útil a otras clases InputStream
SequenceInputStream	Convierte dos o más objetos InputStream en un InputStream único

Tabla 1.2. Tipos de la clase InputStream

Los tipos de **OutputStream** incluyen las clases que deciden dónde irá la salida: a un array de bytes, un fichero o una “tubería”. Se resumen en la siguiente tabla:

CLASE	Función
ByteArrayOutputStream	Crea un espacio de almacenamiento intermedio en memoria. Todos los datos que se envían al flujo se ubican en este espacio
FileOutputStream	Flujo de salida hacia fichero, se usa para leer información de un fichero
PipedOutputStream	Cualquier información que se desee escribir aquí acaba automáticamente como entrada del PipedInputStream asociado. Implementa el concepto de “tubería”.
FilterOutputStream	Proporciona funcionalidad útil a otras clases OutputStream

Tabla 1.3. Tipos de la clase OutputStream

La figura 1.1 muestra la jerarquía de clases para lectura y escritura de flujos de bytes.

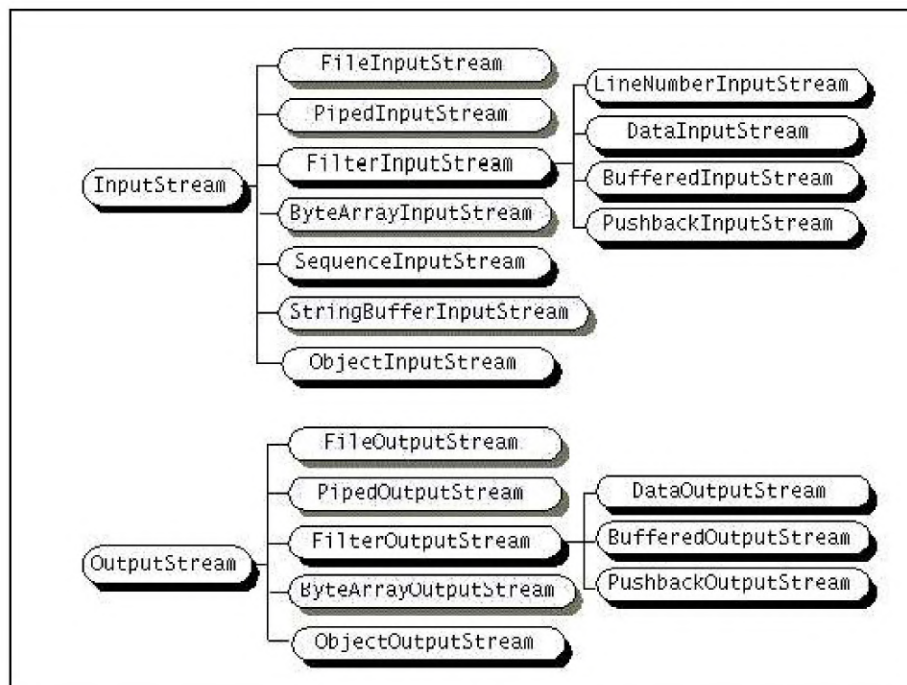


Figura 1.1. Jerarquía de clases para lectura y escritura de bytes

Fuente: Pavón, J. (2008). Entrada y Salida con Java. Madrid: Universidad Complutense Madrid.
Retrieved from <https://www.fdi.ucm.es/profesor/jpavon/poo/2.13.EntradaYSalida.pdf>

4.1.2. Flujos de caracteres (Character streams)

Las clases **Reader** y **Writer** manejan flujos de caracteres Unicode. Hay ocasiones en las que hay que usar las clases que manejan bytes en combinación con las clases que manejan caracteres. Para ello hay clases que sirven para convertir los streams de bytes a streams de caracteres: la clase **InputStreamReader** convierte un **InputStream** en un **Reader** y la clase **OutputStreamReader** convierte un **OutputStream** en un **Writer**.

La siguiente tabla muestra la correspondencia entre las clases de flujos de bytes y de caracteres:

CLASES DE FLUJOS DE BYTES	CLASE CORRESPONDIENTE DE FLUJO DE CARACTERES
InputStream	Reader, convertidor InputStreamReader
OutputStream	Writer, convertidor OutputStreamReader
FileInputStream	FileReader
FileOutputStream	FileWriter
StringBufferInputStream	StringReader
ByteArrayInputStream	CharArrayReader
ByteArrayOutputStream	CharArrayWriter
PipedInputStream	PipedReader
PipedOutputStream	PipedWriter

Tabla 1.4. correspondencia entre las clases de flujos de bytes y de caracteres

La figura 1.2 muestra la jerarquía de clases para lectura y escritura de flujos de caracteres.

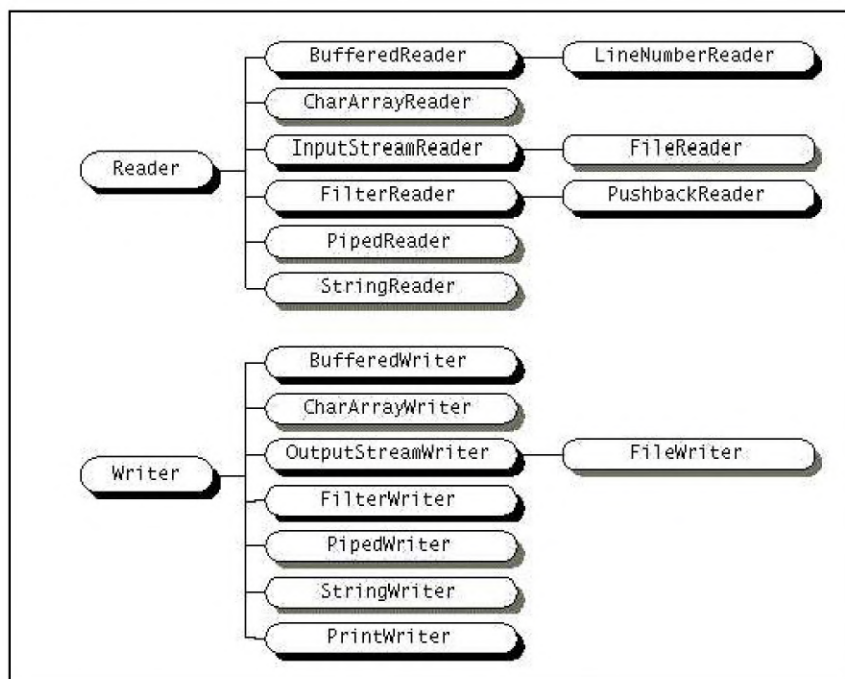


Figura 1.2. Jerarquía de clases para lectura y escritura de flujos de caracteres

Fuente: Pavón, J. (2008). Entrada y Salida con Java. Madrid: Universidad Complutense Madrid.
Retrieved from <https://www.fdi.ucm.es/profesor/jpavon/poo/2.13.EntradaSalida.pdf>

Las clases de flujos de caracteres más importantes son:

- Para acceso a ficheros, lectura y escritura de caracteres de ficheros: **FileReader** y **FileWriter**.
- Para acceso a caracteres **CharArrayReader** y **CharArrayWriter**.
- Para buferización de datos: **BufferedReader** y **BufferedWriter**.

V. FORMAS DE ACCESO A UN FICHERO

Según las características del soporte utilizado, se pueden realizar los siguientes tipos de acceso:

- Acceso secuencial. Consiste en acceder a los registros en el mismo orden en que fueron introducidos. Este tipo de acceso es el obligado en los dispositivos secuenciales, aunque puede utilizarse, si se quiere en los dispositivos direccionables.
- Acceso directo o aleatorio. Permite acceder a cualquier registro directamente. Este acceso sólo es posible en dispositivos direccionables y el tiempo que se tarda en acceder a cualquier registro es constante e igual para todos.

En Java el acceso secuencial más común en ficheros puede ser binario o a caracteres. Para el acceso binario: se usan las clases **FileInputStream** y **FileOutputStream**. Para el acceso a caracteres (texto) se usan las clases **FileReader** y **FileWriter**. En el acceso aleatorio se utiliza la clase **RandomAccessFile**.

5.1 Operaciones sobre ficheros

Las operaciones que se pueden realizar sobre un fichero son:

- Crear un fichero. Crea la estructura de este fichero
- Abrir: abre un fichero si existe para trabajar con él
- Leer: lee el registro activo apuntado por el apuntador
- Escribir: escribe un registro en la posición indicada y avanza el apuntador de registros
- Cerrar: cierra el fichero actual para no perder información
- Eliminar: elimina dicho fichero de la memoria de almacenamiento, liberando el espacio ocupado.
- Renombrar: cambia el nombre de in fichero
- Copiar: copia el contenido de un fichero a una nueva localización.
- Editar: permite modificar el contenido de un fichero

5.1.1 Operaciones sobre ficheros secuenciales

En esta organización, los registros figuran almacenados contiguamente, uno a continuación de otro, siguiendo la secuencia lógica del archivo. Todas las operaciones que se realizan sobre el archivo se hacen según esta secuencia. Esta es la única

organización de archivos susceptibles de ser gestionada en un dispositivo no direccionable. La secuencia en que aparecen los registros en el archivo puede estar determinada por el valor de algún campo o ser simplemente temporal.

Las distintas operaciones o acciones que se pueden realizar sobre archivos con esta organización son:

- **Añadir:** Solo es posible escribir al final del archivo.
- **Consulta o recuperación:** La consulta se realiza en orden secuencial. Esto es, para leer el registro que ocupa la posición “n” en el archivo, es necesario leer previamente los “n-1” anteriores.
- **Actualización:** No es posible realizar fácilmente las operaciones de inserción, modificación o borrado sobre un archivo secuencial. Si se necesita actualizar un archivo con organización secuencial, se debe crear de nuevo el archivo.

Si el archivo se encuentra sobre un soporte direccionable, es posible realizar algunas actualizaciones sobre el archivo secuencial, sin necesidad de crear otro archivo.

La organización secuencial es adecuada para archivos en los que no se realizan procesos interactivos, y en los que en cada proceso se debe acceder a la mayor parte de los registros del archivo.

Presentan la ventaja de aprovechar bien el espacio, ser sencillo de utilizar y de poderse utilizar con dispositivos secuenciales que son baratos.

5.1.2 Operaciones sobre ficheros aleatorios

Son ficheros con organización relativa y clave alfanumérica, por lo que se debe transformar obteniéndose un valor numérico entero que facilite la correspondencia directa entre la clave y la dirección de memoria. En este caso, las direcciones lógicas de almacenamiento, las claves, no coinciden con las direcciones físicas, que son las posiciones de cada registro.

En este tipo de organización, cada dirección física puede ser ocupada por más de un registro, ya que el algoritmo de transformación aplicado a la clave puede generar, a partir de claves diferentes, la misma posición de almacenamiento en memoria.

Las diferentes operaciones a realizar en estos ficheros:

- **Consulta:** Hay que aplicarle a la clave el algoritmo de transformación y posteriormente leer el registro en la posición resultante. Si el registro no se encuentra, se procederá, en función de cómo se haya resuelto, a gestionar la zona de excedentes (la zona donde se ubican registros que no se pueden insertar en posiciones ya ocupadas).

- **Eliminación:** Siempre se realizará un borrado lógico, es decir, el registro no desaparece físicamente del disco. Simplemente, le daremos el valor 0 para que no exista, y 1 si existe.
- **Inserción:** Aplicaremos el algoritmo de transformación y obtendremos la dirección física de almacenamiento. A continuación, leemos el registro para ver si existe. Si no existe, grabaremos dicho registro con la dirección obtenida y si existe, se insertaría en la zona de excedentes.

La organización indirecta es útil para archivos donde los accesos deben realizarse por clave, accediéndose siempre a registros concretos.

VI. CLASES PARA GESTIÓN DE FLUJOS DE DATOS DESDE/HACIA FICHEROS

En Java se puede utilizar dos tipos de ficheros: de texto o binarios y el acceso a los mismos se puede realizar de forma secuencial o aleatoria. Los ficheros de texto están compuestos de caracteres legibles, mientras que los binarios pueden almacenar cualquier tipo de dato (int, float, boolean, etc)

6.1 Ficheros de texto

Los archivos de texto plano son aquellos que están compuestos únicamente por texto sin formato, solo caracteres alfanuméricos.

Para leer un fichero de texto carácter a carácter se debe utilizar la clase Java **FileReader** mientras que para escribir en un fichero de texto utiliza la clase Java **FileWriter**. Al utilizar cualquier clase java para operaciones con ficheros, es necesario que las operaciones que se realicen se encuentren dentro de una estructura **try-catch**. El método constructor de la clase **FileReader** requiere que se indique por parámetro el fichero que se quiere leer. La referencia a dicho fichero se puede realizar con una cadena de caracteres con la ruta y el nombre del mismo. Si el fichero no es encontrado o no es válido, el método produce una excepción **FileNotFoundException**.

La clase **FileReader** dispone del método **read** que permite leer sucesivamente un carácter de un fichero de texto, partiendo desde el inicio del fichero. Retorna un valor entero (int) con número de caracteres leídos. En el momento en que se haya alcanzado el final del fichero (no haya más datos que leer) el método **read** retorna el valor -1.

Método	Función
<code>int read()</code>	Lee un carácter y lo devuelve
<code>int read(char[] buf)</code>	Lee hasta <code>buf.length</code> caracteres de datos de una matriz de caracteres (<code>buf</code>). Los caracteres leídos del fichero se van almacenando en <code>buf</code>
<code>int read(char[] buf, int desplazamiento, int n)</code>	Lee hasta <code>n</code> caracteres de datos de la matriz <code>buf</code> comenzando por <code>buf[desplazamiento]</code> y devuelve el número leído de caracteres

Tabla 1.5. Método `read()` de la clase `FileReader`

Para crear o abrir un fichero se invoca a la clase **File**, se crea el flujo de entrada hacia el fichero con la clase **FileReader** y a continuación se realizan las operaciones de lectura/escritura. Cuando se finaliza de utilizar el fichero se debe cerrar el fichero mediante el método **close**.

El siguiente ejemplo lee cada uno de los caracteres del fichero de texto `fichero1.txt` que se encuentra en la carpeta `C:\AccesoDatos\UD1\Ejercicios` y lo muestra por pantalla. Se captura la excepción **IOException** del método **read** y la excepción **FileNotFoundException** en caso de no existir el fichero. Con el bloque *finally* se asegura que se cierra el fichero pase lo que pase.

```
import java.io.*;
public class LeerFicheroTexto {

    public static void main(String[] args) {
        //declarar fichero
        File fichero = new
File("C:\\AccesoDatos\\UD1\\Ejercicios\\fichero1.txt");
        //Declarar una variable FileReader
        FileReader fr = null;
        try {
            //Abrir el fichero indicado en la variable fichero
            fr = new FileReader(fichero);
            //Se recorre el fichero hasta encontrar el carácter -1
            //que marca el final del fichero
            int i;
            char b[] = new char[10];
            while((i = fr.read(b)) != -1) {
```

```

        System.out.println(b);
    }

    } catch (FileNotFoundException e) {
        //Operaciones en caso de no encontrar el fichero
        System.out.println("Error: Fichero no encontrado");
        //Mostrar el error producido por la excepción
        System.out.println(e.getMessage());
    } catch (IOException e) {
        //Operaciones en caso de error general
        System.out.println("Error de lectura del fichero");
        System.out.println(e.getMessage());
    }
    finally {
        //Operaciones que se harán en cualquier caso. Si hay error o no.
        try {
            //Cerrar el fichero si se ha abierto
            if(fr != null)
                fr.close();
        }
        catch (Exception e) {
            System.out.println("Error al cerrar el fichero");
            System.out.println(e.getMessage());
        }
    }
}

}

}

```

Para poder leer más de un caracter, por ejemplo de 10 en 10:

```

int i;
char b[] = new char[10];
while((i = fr.read(b)) != -1) {
    System.out.println(b);
}

```

FileReader no permite leer líneas completas. **BufferedReader** sí, mediante el método **readLine()**. Devuelve la línea o null si no hay nada que leer o llegamos al final del fichero. También dispone del método **read()** para leer carácter a carácter. El siguiente ejemplo modifica el ejemplo anterior para poder línea por línea.

```

import java.io.*;
public class LeerFicheroTexto {

    public static void main(String[] args) {
        BufferedReader fichero = null;
        String linea;
        try {

```

```

        fichero = new BufferedReader(new
FileReader("C:\\AccesoDatos\\UD1\\Ejercicios\\fichero1.txt"));
        while((linea = fichero.readLine()) != null) {
            System.out.println(linea);
        }

        } catch (FileNotFoundException e) {
            //Operaciones en caso de no encontrar el fichero
            System.out.println("Error: Fichero no encontrado");
            //Mostrar el error producido por la excepción
            System.out.println(e.getMessage());
        }
        catch (IOException e) {
            //Operaciones en caso de error general
            System.out.println("Error de lectura del fichero");
            System.out.println(e.getMessage());
        }
        finally {
            //Operaciones que se harán en cualquier caso. Si hay error o no.
            try {
                //Cerrar el fichero si se ha abierto
                if(fichero != null)
                    fichero.close();
            }
            catch (Exception e) {
                System.out.println("Error al cerrar el fichero");
                System.out.println(e.getMessage());
            }
        }
    }
}

```

Los métodos que proporciona la clase **FileWriter** para escritura son:

Método	Función
void write (int c)	Escribe un caracter
void write(char[] buf)	Escribe un array de caracteres.
void write(char[] buf, int desplazamiento, int n)	Escribe n caracteres de datos de la matriz buf comenzando por buf[desplazamiento]
void write(String str)	Escribe una cadena de caracteres.
void append(char c)	Añade un carácter a un fichero.

Tabla 1.6. Método write() de la clase FileWriter

El metodo **write** también puede lanzar la excepción **IOException**. Con la clase **FileWriter** se crea el flujo de salida hacia el fichero. El siguiente ejemplo muestra cómo escribir en un fichero. Si el fichero no existe lo crea. Si el fichero existe al escribir sobre él, lo sobrescribe. Para que no lo sobrescriba, se debe usar el método

```
FileWriter fic = new FileWriter(fichero, true);
```

```
import java.io.*;
public class EscribirFicheroTexto {

    public static void main(String[] args) {
        //declarar fichero
        File fichero = new
File("C:\\AccesoDatos\\UD1\\Ejercicios\\fichero1.txt");
        //Declarar una variable FileWriter
        FileWriter fw = null;

        String texto = "Probando FileWriter";
        //Convertir el texto en array de caracteres para extraerlos 1 a
1
        char[] cadena = texto.toCharArray();
        try {
            fw = new FileWriter(fichero);
            //Recorre la cadena caracter a caracter
            for(int i=0; i<cadena.length; i++) {
                //Escribe un caracter
                fw.write(cadena[i]);
            }
            //Añadir al final un *
            fw.append('*');
        } catch (IOException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
        finally {
            //Operaciones que se harán en cualquier caso. Si hay error o no.
            try {
                //Cerrar el fichero si se ha abierto
                if(fw != null)
                    fw.close();
            }
            catch (Exception e) {
                System.out.println("Error al cerrar el fichero");
                System.out.println(e.getMessage());
            }
        }
    }
}
```

La clase **BufferedWriter** deriva de la clase **Writer**. Esta clase permite escribir una cadena de caracteres al final de un fichero de texto con el método **write(String)**. En caso de que no exista el fichero, lo crea automáticamente. Si el fichero ya existe, el contenido que tuviera es borrado y se escribe sobre él el nuevo. Tiene el método **newLine()** para saltar de línea.

```
BufferedWriter fichero = new BufferedWriter(new
FileWriter("C:\\AccesoDatos\\UD1\\Ejercicios\\fichero1.txt"));
```

La clase **PrintWriter** también deriva de **Writer** y posee los métodos **print(String)** y **println(String)** para escribir en un fichero. La diferencia es que el segundo introduce un salto de línea:

```
PrintWriter fichero = new PrintWriter(new FileWriter(NombreFichero));
```

6.2 Ficheros binarios

Un fichero binario o de datos está formado por secuencias de bytes. Estos archivos pueden contener datos de tipo básico (int, float, char, etc) y objetos.

Para poder leer el contenido de un fichero binario hay que conocer la estructura interna del fichero, es decir, saber cómo se han escrito: si hay enteros, long, etc. y en qué orden están escritos en el fichero. Si no se conoce su estructura se puede leer byte a byte.

En Java, las dos clases que nos permiten trabajar con ficheros binarios son **FileInputStream** para entrada y **FileOutputStream** para salida.

La forma de crear un fichero binario es igual que la vista con **FileReader** y **FileWriter**.

Los métodos de lectura que proporciona **FileInputStream** devuelven el número de bytes leídos o -1 si se ha llegado al final del fichero:

Método	Función
int read()	Lee un byte y lo devuelve
int read(char[] buf)	Lee hasta buf.length bytes de datos de una matriz de bytes (buf).
int read(char[] buf, int desplazamiento, int n)	Lee hasta n bytes de la matriz buf comenzando por buf[desplazamiento] y devuelve el número leído de bytes

Tabla 1.7. Método read() de la clase FileInputStream

Los métodos de escritura que proporciona **FileOutputStream** son:

Método	Función
<code>void write (int b)</code>	Escribe un byte
<code>void write(byte[] buf)</code>	Escribe buf.length bytes.
<code>void write(byte[] buf, int desplazamiento, int n)</code>	Escribe n bytes a partir de la matriz buf comenzando por buf[desplazamiento]

Tabla 1.8. Método write() de la clase FileOutputStream

Ejemplo escribir y leer bytes en un fichero:

```
import java.io.*;
public class FicheroBinario {

    public static void main(String[] args) {
        //Declarar fichero
        File fichero = new
File("C:\\AccesoDatos\\UD1\\Ejercicios\\ficherob.dat");

        FileOutputStream fileout = null;
        FileInputStream filein = null;
        try {
            //Crear flujo salida hacia el fichero
            fileout = new FileOutputStream(fichero);

            //Crear flujo entrada hacia el fichero
            filein = new FileInputStream(fichero);

            int i;

            //Escribir en fichero
            for(i=1; i<=100; i++) {
                fileout.write(i);
            }

            //Leer fichero
            while((i = filein.read()) != -1) {
                System.out.println(i);
            }
        } catch (FileNotFoundException e) {
            //Operaciones en caso de no encontrar el fichero
            System.out.println("Error: Fichero no encontrado");
            //Mostrar el error producido por la excepción
            System.out.println(e.getMessage());
        } catch (IOException e) {
```

```
        //Operaciones en caso de error general
        System.out.println("Error");
        System.out.println(e.getMessage());
    }
    finally {
        //Operaciones que se harán en cualquier caso. Si hay error o no.
        try {
            if (fileout != null) {
                fileout.close();
            }
            if (filein != null) {
                filein.close();
            }
        }
        catch (Exception e) {
            System.out.println("Error al cerrar el fichero");
            System.out.println(e.getMessage());
        }
    }
}
}
```

Para añadir bytes al final del fichero se coloca un segundo parámetro con valor **true** al constructor de **FileOutputStream**:

```
FileOutputStream fileout = new FileOutputStream(fichero, true);
```

A partir de un objeto **FileOutputStream** se puede crear un objeto **DataOutputStream**, que proporciona métodos para escribir datos de tipo primitivo en el fichero.

Para crear un objeto **DataOutputStream**:

```
File fichero = new File("fichero.dat");
FileOutputStream fileout = new FileOutputStream(fichero);
DataOutputStream dataOS = new DataOutputStream(fileout);
```

La clase proporciona métodos `writeXxx()`, que son variaciones del método **write()** de la clase base para escribir datos, donde Xxx es el nombre del tipo primitivo. Lanzas una **IOException**.

MÉTODOS PARA ESCRITURA
<code>void writeBoolean(boolean v);</code>
<code>void writeByte(int v);</code>
<code>void writeUTF(String s);</code>
<code>void writeShort(int v);</code>
<code>void writeChars(String s);</code>
<code>void writeChar(int v);</code>
<code>void writeInt(int v);</code>
<code>void writeLong(long v);</code>
<code>void writeFloat(float v);</code>
<code>void writeDouble(double v);</code>

Tabla 1.9. Variantes del método `write()` de la clase `DataOutputStream`

A partir de un objeto **`FileInputStream`** se puede crear un objeto **`DataInputStream`**, que proporciona métodos para escribir datos de tipo primitivo en el fichero.

Para crear un objeto **`DataInputStream`**:

```
File fichero = new File("fichero.dat");
FileInputStream filein = new FileInputStream(fichero);
DataInputStream dataIS = new DataInputStream(filein);
```

La clase proporciona métodos `readXxx()`, que son variaciones del método **`read()`** de la clase base para leer datos, donde `Xxx` es el nombre del tipo primitivo. Lanzas una `IOException`.

MÉTODOS PARA LECTURA
<code>boolean readBoolean();</code>
<code>byte readByte();</code>
<code>int readUnsignedByte();</code>
<code>short readShort();</code>
<code>int readUnsignedShort();</code>
<code>char readChar();</code>
<code>int readInt();</code>
<code>String readUTF();</code>
<code>long readLong();</code>
<code>float readFloat();</code>
<code>double readDouble();</code>

Tabla 1.11. Variantes del método read() de la clase DataInputStream

6.3 Objetos en ficheros binarios

Hasta ahora se han guardado los tipos de datos primitivos, pero si se quiere guardar en un fichero un objeto de tipo cliente con varios atributos (nombre, DNI, dirección, teléfono, etc.) se debe guardar cada atributo que forma parte del objeto por separado. En Java existe un mecanismo conocido como serialización de objetos, que permite convertir cualquier objeto cuya clase implemente el interface **Serializable** en una secuencia de bytes que pueden ser posteriormente leídos para restaurar el objeto original. De esta forma no hay que preocuparse en absoluto de las diferentes representaciones de datos.

Para personalizar la serialización, es necesario definir dos funciones miembros **writeObject** (que puede lanzar `IOException`) y **readObject** (que puede lanzar `IOException` y `ClassNotFoundException`). El primero, controla que información es enviada al flujo de salida. La segunda, lee la información escrita por `writeObject`.

El método **readObject** ha de leer todo lo que se ha escrito con **writeObject** en el mismo orden en el que se ha escrito. Además, puede realizar otras tareas necesarias para actualizar el estado del objeto. Leerá hasta el final del fichero cuando se lanza la excepción **EOFException**.

Dos flujos de datos **ObjectInputStream** y **ObjectOutputStream** están especializados en la lectura y escritura de objetos. El comportamiento de estos dos flujos es similar a sus correspondientes que procesan flujos de datos primitivos **DataInputStream** y **DataOutputStream** ya vistos en esta unidad.

A continuación se muestra la clase *Cliente* que implementa el interfaz **Serializable** para poder ser escrita y leída como objeto en un fichero binario:

```
import java.io.Serializable;

public class Cliente implements Serializable {
    private String nombre;
    private int edad;

    public Cliente(String nombre,int edad) {
        this.nombre=nombre;
        this.edad=edad;
    }
    public Cliente() {
        this.nombre=null;
    }
    public void setNombre(String nom){nombre=nom;}
    public void setEdad(int ed){edad=ed;}

    public String getNombre(){return nombre;}
    public int getEdad(){return edad;}
}
```

El siguiente ejemplo escribe objetos *Cliente* en un fichero. Para ellos crea un flujo de salida a disco con **FileOutputStream**. Después, se crea otro flujo de salida **ObjectOutputStream** para procesar los datos y por último vinculándolo al fichero de **FileOutputStream**:

```
import java.io.*;

public class EscribirFichObject {
    public static void main(String[] args) throws IOException {

        Cliente cliente;//definir variable cliente

        File fichero = new
File("C:\\AccesoDatos\\UD1\\Ejercicios\\FichCliente.dat"); //declara el
fichero
```

```

    FileOutputStream fileout = new FileOutputStream(fichero,true); //crea el
    flujo de salida
    //conecta el flujo de bytes al flujo de datos
    ObjectOutputStream dataOS = new ObjectOutputStream(fileout);

    String nombres[] = {"Antonio","Laura","Miguel","Alberto","Rebeca",
    "","Sara"};

    int edades[] = {18,20,21,33,25,42};

    for (int i=0;i<edades.length; i++){ //recorrer los arrays
        cliente= new Cliente(nombres[i],edades[i]); //crea el cliente
        dataOS.writeObject(cliente); //escribe el cliente en el fichero
    }
    dataOS.close(); //cerrar stream de salida
}
}

```

Para leer objetos *Cliente* del fichero se necesita crear un flujo de entrada a disco con **FileInputStream**, otro flujo de salida **ObjectInputStream** para procesar los datos vinculándolo al fichero de **FileInputStream**:

```

import java.io.*;

public class LeerFichObject {
    public static void main(String[] args) throws IOException,
    ClassNotFoundException{
        Cliente cliente; //definir la variable cliente
        File fichero = new
        File("C:\\AccesoDatos\\UD1\\Ejercicios\\FichCliente.dat");//declara el
        fichero
        FileInputStream filein = new FileInputStream(fichero);//crea el flujo de
        entrada
        //conecta el flujo de bytes al flujo de datos
        ObjectInputStream dataIS = new ObjectInputStream(filein);
        int i=1;
        try {
            while (true) { //lectura del fichero
                cliente= (Cliente) dataIS.readObject(); //leer un Cliente
                System.out.println("Nombre: " + cliente.getNombre() + ", edad: " +
                cliente.getEdad());
                System.out.println(i);i++;
            }
        }catch (EOFException eo) {}
        catch (StreamCorruptedException x) {}
        dataIS.close(); //cerrar stream de entrada
    }
}

```

6.3.1. Problemas con los ficheros de objetos

Existe un problema con los ficheros de objetos. Al crear un fichero de objetos se crea una cabecera inicial con información y a continuación se añaden los objetos. Si se vuelve a utilizar de nuevo el fichero para añadir más registros, se crea una nueva cabecera a la que se le añaden los objetos. El problema surge cuando al leer el fichero la lectura se encuentra con la segunda cabecera, surge la excepción **StreamCorruptedException** y no se pueden leer más objetos.

Cada vez que se crea un nuevo objeto de la clase **ObjectOutputStream** se añade la cabecera. Para evitarlo, se debe redefinir la clase **ObjectOutputStream** a partir de una nueva clase que la herede, y redefinir el método **writeStreamHeader()**, encargado de escribir las cabeceras, para que ese método no haga nada.

```
import java.io.IOException;
import java.io.ObjectOutputStream;
import java.io.OutputStream;

public class MiObjectOutputStream extends ObjectOutputStream{

    protected MiObjectOutputStream(OutputStream out) throws IOException,
    SecurityException {
        super(out);
    }

    public MiObjectOutputStream() throws IOException, SecurityException {
        super();
    }

    //Redefinición el método para evitar escribir la cabecera y que no
    haga nada
    @Override
    protected void writeStreamHeader() throws IOException {

    }

}
```

Dentro del programa principal, se pregunta si el fichero ya existe y si existe, se crea el objeto con la clase redefinida. Si no existe, se crea con la clase **ObjectOutputStream**:

```
File fichero = new File("C:\\AccesoDatos\\UD1\\Ejercicios\\fichero.dat");
ObjectOutputStream dataOS;
if(!fichero.exists()) {
    //Si el fichero no existe crea un ObjectOutputStream
    FileOutputStream fileout;
    fileout = new FileOutputStream(fichero);
    dataOS = new ObjectOutputStream(fileout);
}
else {
    //Si ya existe se invoca a la clase redefinida (sin hacer nada)
    dataOS = new MiObjectOutputStream(new
    FileOutputStream(fichero,true));}
```

6.4 Ficheros de acceso aleatorio

Todas las operaciones sobre ficheros vistas hasta ahora se realizaban de forma secuencial, tanto para lectura como de escritura. La clase Java **RandomAccessFile** se utiliza para acceder a un fichero de forma aleatoria.

Los constructores de la clase son:

- **RandomAccessFile(String path, String modo):** escribiendo el nombre del fichero incluido el path.
- **RandomAccessFile(File objetoFile, String modo):** asociando un objeto File con el fichero.

Lanzan una excepción **FileNotFoundException**.

El argumento *modo* indica el modo de acceso en el que se abre el fichero.

Los valores permitidos para este parámetro son:

Modo	Significado
r	Abre el fichero en modo solo lectura. El fichero debe existir. Una operación de escritura en este fichero lanzará una excepción IOException .
rw	Abre el fichero en modo lectura y escritura. Si el fichero no existe se crea.

Tabla 1.12. Modos de acceso en el que se abre el fichero.

Por ejemplo, para abrir un fichero aleatorio **para lectura**, se abre el fichero clientes.dat usando el primer constructor:

```
RandomAccessFile fichero = new RandomAccessFile  
("C:\\AccesoDatos\\UD1\\Ejercicios\\clientes.dat", "r");
```

Sin embargo, para abrir un fichero aleatorio **para lectura/escritura**, se abre el fichero clientes.dat usando el segundo constructor. Si el fichero no existe, se crea.

```
File f = new File ("C:\\AccesoDatos\\UD1\\Ejercicios\\clientes.dat");  
RandomAccessFile fichero = new RandomAccessFile(f, "rw");
```

Para acceder de forma aleatoria a los datos contenidos en el fichero, la clase **RandomAccessFile** dispone de varios métodos. Entre ellos:

- **long getFilePointer():** Devuelve la posición actual del puntero del fichero. Indica la posición (en bytes) donde se va a leer o escribir.
- **long length():** Devuelve la longitud del fichero en bytes.
- **void seek(long pos):** Coloca el puntero del fichero en una posición pos determinada desde el comienzo del mismo. La posición 0 indica el principio del fichero. La posición length() indica el final del fichero.
- **int skipBytes(int desplazamiento):** Desplaza el puntero desde la posición actual el número de bytes indicados en desplazamiento.

Los métodos de lectura/escritura que dispone la clase **RandomAccessFile** son:

- **public int read():** Devuelve el byte leído en la posición marcada por el puntero. Devuelve -1 si alcanza el final del fichero. Se debe utilizar este método para leer los caracteres de un fichero de texto.
- **public final String readLine():** Devuelve la cadena de caracteres que se lee, desde la posición marcada por el puntero, hasta el siguiente salto de línea que se encuentre.
- **public xxx readXxx():** Hay un método **read** para cada tipo de dato básico (readChar, readInt, readDouble, readBoolean, etc.)
- **public void write(int b):** Escribe en el fichero el byte indicado por parámetro. Se debe utilizar este método para escribir caracteres en un fichero de texto.
- **public final void writeBytes(String s):** Escribe en el fichero la cadena de caracteres indicada por parámetro.
- **public final void writeXxx(argumento):** También existe un método **write** para cada tipo de dato básico (writeChar, writeInt, writeDouble, writeBoolean, etc.)

A continuación se muestra un ejemplo completo para poder insertar datos de empleados (identificador de empleado, apellido, número de departamento y salario) en un fichero aleatorio. Los datos se obtienen de diferentes arrays, salvo el identificador de empleado que coincide con el índice con el que se recorren los arrays.

Los tipos que se insertan y su tamaño en bytes son los siguientes:

1. Se inserta un entero, que es el identificador. Ocupa **4 bytes**.
2. Se inserta el apellido que es una cadena de 10 caracteres. Java utiliza caracteres UNICODE, por lo que cada carácter de la cadena ocupa 2 bytes. Por lo tanto, el apellido ocupa **20 bytes**.
3. Se inserta un entero que es el departamento. Ocupa **4 bytes**.
4. Se inserta un tipo Double que es el salario. Ocupa **8 bytes**.

Con esto, la longitud del registro de cada empleado es **36 bytes (4 + 20 + 4 + 8)**

```
import java.io.*;
public class EscribirFichAleatorio {
    public static void main(String[] args) throws IOException {
        File fichero = new
File("C:\\AccesoDatos\\UD1\\Ejercicios\\AleatorioEmple.dat");
        //declara el fichero de acceso aleatorio
        RandomAccessFile file = new RandomAccessFile(fichero, "rw");
        //arrays con los datos
        String apellido[] = {"DIEZ", "MEDRANO", "GARCIA", "ARANZUBIA",
                             "MARTINEZ", "JIMENEZ", "NALDA"}; //apellidos
        int dep[] = {10, 20, 30, 40, 10, 20, 30}; //departamentos
        Double salario[]={1700.45, 600.60, 2000.0, 750.65,
                           1200.10, 1435.67, 900.0}; //salarios

        StringBuffer buffer = null; //buffer para almacenar apellido
        int n=apellido.length; //numero de elementos del array

        for (int i=0; i<n; i++){ //recorro los arrays
            file.writeInt(i+1); //uso i+1 para identificar empleado
            buffer = new StringBuffer( apellido[i] );
            buffer.setLength(10); //10 caracteres para el apellido
            file.writeChars(buffer.toString()); //insertar apellido
            file.writeInt(dep[i]); //insertar departamento
            file.writeDouble(salario[i]); //insertar salario
        }
        file.close(); //cerrar fichero
    }
}
```

El siguiente ejemplo toma el fichero del ejemplo anterior y visualiza todos los registros. El posicionamiento para empezar a recorrer los registros empieza en 0, por lo que para recuperar los siguientes registros hay que sumar 36 (tamaño del registro) a la variable utilizada para el posicionamiento:

```
import java.io.*;
public class LeerFichAleatorio {
    public static void main(String[] args) throws IOException {
        File fichero = new
File("C:\\AccesoDatos\\UD1\\Ejercicios\\AleatorioEmple.dat");
        //declara el fichero de acceso aleatorio
        RandomAccessFile file = new RandomAccessFile(fichero, "r");
        //
        int id, dep, posicion;
        Double salario;
        char apellido[] = new char[10], aux;

        posicion=0; //Situarse en el principio

        for(;;){ //recorrer fichero
            file.seek(posicion); //Posicionarse en posicion
            id=file.readInt(); // obtener id de empleado
            for (int i = 0; i < apellido.length; i++) {
                aux = file.readChar();//recorrer uno a uno los caracteres del
apellido
                apellido[i] = aux; //guardar en el array
            }
            String apellidos= new String(apellido);//convertir a String el array
            dep=file.readInt();//obtener departamento
            salario=file.readDouble(); //obtener salario

            System.out.println("ID: " + id + ", Apellido: "+ apellidos +
                                ", Departamento: "+dep + ", Salario: " +
salario);
            posicion= posicion + 36; // posicionarse para el sig empleado
                                //Cada empleado ocupa 36 bytes (4+20+4+8)
            //Si se han recorrido todos los bytes salir del for
            if (file.getFilePointer()==file.length())break;

        }//fin bucle for
        file.close(); //cerrar fichero
    }
}
```

Para consultar un empleado determinado se accede directamente a través de su identificador.

```
int identificador = 5;
    //Calcular donde se encuentra el registro
    posicion = (identificador - 1) * 36;
    if(posicion >= fichero.length()) {
        System.out.printf("ED: %d, NO EXISTE EMPLEADO...", identificador);
    }
    else {
        fichero.seek(posicion);    //Posicionarse sobre el registro
        buscado
        id = file.readInt(); //Obtener ID empleado
        //Obtener resto de datos. Ejemplo anterior
    }
}
```

Para añadir registros a partir del último insertado se posiciona el puntero al final del fichero:

```
long posicion = fichero.length();

fichero.seek(posicion);
```

Para insertar un nuevo registro se calcula la posición donde irá el registro dentro del fichero $(identificador - 1) * 36$ bytes:

```
int id = 20;
StringBuffer buffer = null;    //buffer para almacenar
String apellido = "CRUZ";
int departamento = 10;
Double salario = 1500.87;

long posicion = (id - 1) * 36; //calcular la posicion

fichero.seek(posicion);    //posicionarse
fichero.writeInt(id);    //insertar id cliente
buffer = new StringBuffer(apellido);
buffer.setLength(10);    //10 caracteres para el apellido
fichero.writeChars(buffer.toString()); //insertar apellido
fichero.writeInt(departamento); //insertar departamento
fichero.writeDouble(salario);    //insertar salario

fichero.close();    //cerrar fichero
```

VII. TRABAJO CON FICHEROS XML: ANALIZADORES SINTÁCTICOS (PARSER) Y VINCULACIÓN (BINDING)

XML es un lenguaje de marcado similar a HTML. Significa *Extensible Markup Language* (Lenguaje de Marcado Extensible) y es una especificación de W3C como lenguaje de marcado de propósito general. Esto significa que, a diferencia de otros lenguajes de marcado, XML no está predefinido, por lo que se debe definir etiquetas propias. Estas etiquetas se delimitan mediante los símbolos menor que < y mayor que >. Las etiquetas se cierran indicando </ seguido del nombre de la etiqueta que se está cerrando. Cada marca tiene un nombre y puede tener 0 o más atributos. El propósito principal del lenguaje es compartir datos a través de diferentes sistemas, como Internet. Un ejemplo de fichero XML sencillo sería la siguiente estructura:

```
<?xml version="1.0"?>
<clients>
  <person>
    <id>1</id>
    <nombre> Pedro </nombre>
    <departamento> 10 </departamento>
  </person>
  <person>
    <id>2</id>
    < nombre > Homer </ nombre >
    <departamento> 20 </departamento>
  </person>
  <person>
    <id>3</id>
    <nombre> Sara </nombre >
    <departamento> 30 </departamento>
  </person>
</clients>
```

Hay muchos lenguajes basados en XML, algunos ejemplos son XHTML, MathML, SVG, XUL, XBL, RSS, y RDF. También se puede crear uno propio.

XML es un formato que tiene cada vez más importancia para generar documentos empresariales por la facilidad de estructurar los datos de la forma más apropiada para la empresa. Un documento XML se parece mucho a una pequeña base de datos, con la ventaja de que es muy fácil darle formato de salida por pantalla o impresión. De hecho, se puede usar para proporcionar datos a una base de datos o para almacenar copias de partes del contenido de la BD. También se usa para escribir ficheros de configuración o en el protocolo SOAP (Simple Object Access Protocol), para ejecutar comandos en servidores remotos; la información enviada al servidor remoto y el resultado de la ejecución del comando se envían en ficheros XML.

En principio XML se puede escribir desde cualquier editor de texto plano (como el bloc de notas de Windows o el editor de Linux). Pero es más interesante hacerlo con un editor que reconozca el lenguaje y que además marque los errores en el mismo. De hecho el software necesario es el siguiente:

- Un **editor de texto plano** para escribir el código XML.
- Un **analizador sintáctico o parser**. Programa capaz de entender y validar el lenguaje XML.
- Un **procesador XML** que sea capaz de producir un resultado visual sobre el documento XML. Un simple navegador puede hacer esta función, pero cuando se aplican formatos visuales sobre el documento XML (como los creados mediante XSL) entonces hace falta un software especial que convierta los datos a la forma final visible por el usuario. Lee los documentos y proporciona acceso a su contenido y estructura.

XML posee un gran número de tecnologías para dar funcionalidad, presentación o integración con otros lenguajes. Dos de los procesadores más empleados son:

- **DOM:** Document Object Model, almacena toda la estructura del documento en memoria en forma de árbol con nodos padre, nodos hijo y nodos finales (los que no tienen descendientes). Una vez creado el árbol, se van recorriendo los diferentes nodos y se analiza a qué tipo particular pertenecen. Tiene su origen en el W3C. Este tipo de procesamiento necesita más recursos de memoria y tiempo sobre todo si los ficheros XML a procesar son bastante grandes y complejos.
- **SAX:** Simple API for XML, lee un fichero XML de forma secuencial y produce una secuencia de eventos (comienzo/fin del documento, comienzo/fin de una etiqueta, etc) en función de los resultados de la lectura. Este tipo de procesamiento prácticamente no consume memoria, pero, por otra parte, impide tener una visión global del documento por el que navegar.

7.1 Acceso a ficheros XML con DOM

A la hora de leer un archivo XML a través de DOM hay que instanciar una serie de clases antes de poder tratar el fichero. Estas clases las proporcionan los paquetes **org.w3c.dom** y **javax.xml.parsers** (proporciona un par de clases abstractas que toda implementación DOM para Java debe extender). Contiene dos clases fundamentales: **DocumentBuilderFactory** y **DocumentBuilder**.

DOM no define ningún mecanismo para generar un fichero XML a partir de un árbol DOM. Para eso se usa el paquete **javax.xml.transform** que permite especificar una fuente y un resultado.

El DOM define, entre otras, las siguientes interfaces de objetos XML:

- **Document:** Proporciona información del documento. Permite crear nuevos nodos en el documento.
- **Element:** Expone propiedades y métodos para manipular los elementos del documento y sus atributos.
- **Node:** Representa a cualquier nodo del documento.
- **NodeList:** Contiene una lista con los nodos hijos de un nodo a los que se puede acceder por medio de un índice.
- **Attr:** Permite acceder a los atributos de un nodo.
- **CharacterData:** Proporciona atributos y métodos para manipular los datos de caracteres presentes en el documento.
- **DocumentType:** Proporciona información contenida en la etiqueta `<!DOCTYPE>`.

Ejemplo. Crear un fichero XML a partir del fichero aleatorio de empleados creado en el epígrafe anterior. Primero instanciar las clases **DocumentBuilderFactory** que define un API de fábrica que permite a las aplicaciones obtener un analizador que produce árboles DOM, **DocumentBuilder**, que obtiene una instancia de documento DOM y permite obtener un Documento XML y **Document**. Definir toda la estructura del archivo (siempre dentro de un bloque **try-catch** -> **ParserConfigurationException**) y por último instanciar las clases **TransformerFactory**, **Transformer**, **DOMSource** y **StreamResult** para crear el archivo.

Crear un documento vacío de nombre *document* con el nodo raíz de nombre *Empleados* y asignar la versión del XML. La interfaz **DOMImplementation** permite crear objetos **Document** con nodo raíz.

A continuación recorrer el el fichero y por cada registro crear un nodo empleado con 4 hijos (*id*, *apellido dep* y *salario*). Cada nodo hijo tendrá su valor. Para crear los elementos se usa el método **createElement(String)** mientras que para añadir su valor se usa el método **createTextNode(String)**

El código completo para crear el fichero **xml** es el siguiente:

```
import org.w3c.dom.*;
import javax.xml.parsers.*;
import javax.xml.transform.*;
import javax.xml.transform.dom.*;
import javax.xml.transform.stream.*;
import java.io.*;

public class CrearEmpleadoXml {
    public static void main(String args[]) throws IOException {
        File fichero = new
File("C:\\AccesoDatos\\UD1\\Ejercicios\\AleatorioEmple.dat");
        RandomAccessFile file = new RandomAccessFile(fichero, "r");

        // situarse al principio del fichero
        int id, dep, posicion = 0;
        Double salario;
        char apellido[] = new char[10], aux;

        DocumentBuilderFactory factory =
DocumentBuilderFactory.newInstance();

        try {
            DocumentBuilder builder = factory.newDocumentBuilder();
            DOMImplementation implementation =
builder.getDOMImplementation();
            Document document = implementation.createDocument(null,
"Empleados", null);
            document.setXmlVersion("1.0");

            for (;;) {
                file.seek(posicion); // posicionarse en el fichero
                id = file.readInt(); // obtener id de empleado
                for (int i = 0; i < apellido.length; i++) {
                    aux = file.readChar();
                    apellido[i] = aux;
                }
                String apellidos = new String(apellido);
                dep = file.readInt();
                salario = file.readDouble();

                // id validos a partir de 1
                if (id > 0) {
                    // nodo empleado
                    Element raiz = document.createElement("empleado");
                    document.getDocumentElement().appendChild(raiz);
                    // añadir ID
```

```

        CrearElemento("id", Integer.toString(id), raiz,
document);
        // Apellido
        CrearElemento("apellido", apellidos.trim(), raiz,
document);
        // añadir DEP
        CrearElemento("dep", Integer.toString(dep), raiz,
document);
        // añadir salario
        CrearElemento("salario", Double.toString(salario),
raiz, document);
    }
    posicion = posicion + 36; // me posiciono para el
sig empleado
        if (file.getFilePointer() == file.length())
            break;

    } // fin del for que recorre el fichero

    Source source = new DOMSource(document);
    Result result = new StreamResult(new
java.io.File("Empleados.xml"));
    Transformer transformer =
TransformerFactory.newInstance().newTransformer();
    transformer.transform(source, result);

    } catch (Exception e) {
        System.err.println("Error: " + e);
    }

    file.close(); // cerrar fichero
} // fin de main

// Método que realiza la inserción de los datos del empleado
static void CrearElemento(String datoEmple, String valor, Element
raiz, Document document) {
    Element elem = document.createElement(datoEmple);
    Text text = document.createTextNode(valor); // dar valor
    raiz.appendChild(elem); // pegar el elemento hijo a la raiz
    elem.appendChild(text); // pegar el valor
}

} // fin de la clase

```

Para leer un archivo XML primero se instancia la clase `File` para cargar el fichero. Posteriormente y ya dentro de un **try-catch** (para tratar las excepciones) parsear el fichero con estas clases: **DocumentBuilderFactory**, **DocumentBuilder** y **Document**.

Una vez hecho todo esto ya se puede leer el archivo xml además de usar otros métodos como los ejemplos de aquí abajo.

Método	Descripción
<code>getDocumentElement()</code>	Accede al nodo raíz del documento
<code>normalize()</code>	Elimina nodos vacíos y combina adyacentes en caso de que los hubiera

Tabla 1.13. Métodos para trabajar con ficheros XML.

Estos métodos se pueden combinar para normalizar el archivo XML

```
document.getDocumentElement().normalize();
```

Siguiendo dentro del **try-catch**, utilizar la clase **NodeList** para almacenar el elemento que se le indique como parámetro.

```
NodeList empleados = document.getElementsByTagName("empleado");
System.out.printf("Nodos empleado a recorrer: %d %n",
    empleados.getLength());
```

El código al completo es el siguiente:

```
import java.io.File;
import javax.xml.parsers.*;
import org.w3c.dom.*;

public class LecturaEmpleadoXml {
    public static void main(String[] args) {

        DocumentBuilderFactory factory =
            DocumentBuilderFactory.newInstance();

        try {
            DocumentBuilder builder = factory.newDocumentBuilder();
            Document document = builder.parse(new
File("C:\\AccesoDatos\\UD1\\Ejercicios\\Empleados.xml"));
            document.getDocumentElement().normalize();

            System.out.printf("Elemento raíz: %s %n",
                document.getDocumentElement().getNodeName());
            //crea una lista con todos los nodos empleado
            NodeList empleados = document.getElementsByTagName("empleado");
            System.out.printf("Nodos empleado a recorrer: %d %n",
                empleados.getLength());

            //recorrer la lista
```

```
for (int i = 0; i < empleados.getLength(); i++) {  
    //obtener un nodo empleado  
    Node emple = empleados.item(i);  
    if (emple.getNodeType() == Node.ELEMENT_NODE) { //tipo de nodo  
        //obtener los elementos del nodo  
        Element elemento = (Element) emple;  
        System.out.printf("ID = %s %n",  
            elemento.getElementsByTagName("id").  
                item(0).getTextContent());  
        System.out.printf(" * Apellido = %s %n",  
            elemento.getElementsByTagName("apellido").  
                item(0).getTextContent());  
        System.out.printf(" * Departamento = %s %n",  
            elemento.getElementsByTagName("dep").  
                item(0).getTextContent());  
        System.out.printf(" * Salario = %s %n",  
            elemento.getElementsByTagName("salario").  
                item(0).getTextContent());  
    }  
}  
} catch (Exception e)  
{e.printStackTrace();}  
  
} //fin de main  
} //fin de la clase
```

7.2 Acceso a ficheros XML con SAX

El API **SAX** es un API orientada a eventos. A diferencia del DOM, no conlleva la generación de estructuras internas. Va cargando en memoria de forma secuencial, por lo que a diferencia de DOM, no carga todo el documento en la memoria. La mayoría de los procesadores soportan esta API. El API SAX, define una serie de interfaces, cuyos métodos son controladores de evento. Cuando se analiza un documento, el procesador notifica a los controladores de evento, los distintos sucesos que tienen lugar (principio de una etiqueta, instrucción de procesamiento,...) Todas las operaciones de la aplicación se ejecutan durante la fase de análisis.

En la siguiente tabla se puede ver un ejemplo para comprender el método asociado a cada evento del documento:

Documento XML (alumnos.xml)	Métodos asociados a eventos del documento
<pre><?xml version="1.0"?> <listadealumnos> <alumno> <nombre> Juan </nombre> <edad> 19 </edad> </alumno> <alumno> <nombre> Maria </nombre> <edad> 20 </edad> </alumno> </listadealumnos></pre>	<pre>startDocument() startElement() startElement() startElement() characters() endElement() startElement() characters() endElement() endElement() startElement() startElement() characters() endElement() startElement() characters() endElement() endElement() endElement() endDocument()</pre>

Las clases más importantes a tener en cuenta son:

- **Interface *org.xml.sax.XMLReader*:** Es la interfaz que debe implementar un lector de XML. Cada vez que el **XMLReader** encuentra el principio del archivo, el final, un elemento, un caracter especial, un espacio, etc, lo notifica al **ContentHandler** asociado y cada vez que encuentra un error se lo notifica al **ErrorHandler** asociado.
- **Interface *org.xml.sax.ContentHandler*:** La clase que la implementa obtiene la capacidad de recibir todas las notificaciones de contenido de un **XMLReader**. La forma de asociar éste a un **XMLReader** es mediante el método **setContentHandler**.
- **Interface *org.xml.sax.ErrorHandler*:** La clase que la implementa obtiene la capacidad de recibir todas las notificaciones de error producidas por un **XMLReader**. La forma de asociar este a un **XMLReader** es mediante el método **setErrorHandler**. En caso de que no exista ningún **ErrorHandler** asociado al **XMLReader**, éste último no reportará ningún error, salvo las excepciones **SAXParseException**.
- **Clase *org.xml.sax.helpers.DefaultHandler*:** Clase que implementa tanto a **ContentHandler** como a **ErrorHandler**. Provee implementaciones por defecto para todos sus métodos. De esta clase extenderemos para poder crear nuestro parser de XML.

- Clase `javax.xml.parsers.SAXParserFactory` y `javax.xml.parsers.SAXParser`: Clase que permiten la creación de una fábrica de parsers SAX y luego de un parser SAX sobre el que se puede construir el `XMLReader`.

Ejemplo.

```
import java.io.IOException;
import java.util.logging.Level;
import java.util.logging.Logger;
import javax.xml.parsers.ParserConfigurationException;
import javax.xml.parsers.SAXParser;
import javax.xml.parsers.SAXParserFactory;
import org.xml.sax.Attributes;
import org.xml.sax.InputSource;
import org.xml.sax.SAXException;
import org.xml.sax.XMLReader;
import org.xml.sax.helpers.DefaultHandler;

/**
 *
 * @author Lorena
 */
public class Ficheros_XML_SAX {

    public static void main(String[] args) {
        try {
            //creamos un xmlreader
            SAXParserFactory fabrica = SAXParserFactory.newInstance();
            SAXParser parser = fabrica.newSAXParser();
            XMLReader lector = parser.getXMLReader();

            GestionContenido gestor= new GestionContenido();
            lector.setContentHandler(gestor);
            InputSource file = new
InputSource("C:\\AccesoDatos\\UD1\\Ejercicios\\Empleados.xml");
            lector.parse(file);

        } catch (ParserConfigurationException ex) {
            System.out.println("Problema al crear el XMLreader");
        }
        Logger.getLogger(Ficheros_XML_SAX.class.getName()).log(Level.SEVERE,
        null, ex);
    } catch (SAXException ex) {
```

```

        System.out.println("Problema al llamar a los métodos");

Logger.getLogger(Ficheros_XML_SAX.class.getName()).log(Level.SEVERE,
null, ex);
    } catch (IOException ex) {
        System.out.println("Problema de E/S");

Logger.getLogger(Ficheros_XML_SAX.class.getName()).log(Level.SEVERE,
null, ex);
    }
}
}

class GestionContenido extends DefaultHandler{

    public GestionContenido() {
        super();
    }

    @Override
    public void characters(char[] ch, int start, int length) throws
SAXException {
        //creamos un string con la cadena de caracteres pasada
        String character = new String(ch, start, length);
        //quitamos saltos de línea
        character = character.replaceAll("[\t\n]", "");
        System.out.println("Caracteres: "+character);
    }

    @Override
    public void endElement(String uri, String localName, String qName)
throws SAXException {
        System.out.println("Fin del elemento: "+qName);
    }

    @Override
    public void startElement(String uri, String localName, String
qName, Attributes attributes) throws SAXException {
        System.out.println("Comienzo del elemento: "+qName);
    }

    @Override
    public void endDocument() throws SAXException {
        System.out.println("Final del documento XML");
    }
}

```

```
}  
  
@Override  
public void startDocument() throws SAXException {  
    System.out.println("Comienzo del documento XML");  
}  
}
```

7.3 Serialización de objetos a XML

Una forma sencilla de serializar objetos Java a XML y viceversa, es utilizando la librería **XStream**. Para poder utilizarla hay que descargar previamente los jar desde el sitio Web: <http://x-stream.github.io/download.html>. En el momento de la redacción de este documento se descargó el fichero **xstream-distribution-1.4.20-bin**. Descomprimir y buscar el JAR **xstream-1.4.20.jar** que se encuentra en la carpeta *lib*, y el fichero **kxml2-min-2.3.0.jar** de la carpeta *lib\xstream*. Añadir ambos ficheros al proyecto o al CLASSPATH. Lo vamos a añadir a las dependencias (Add dependency).

Partiendo del fichero *FichCliente.dat* o *fichObjetos.dat* utilizado en epígrafes anteriores, se crea una lista de objetos *Cliente* que se convertirá en un fichero de datos XML.

```
import java.util.ArrayList;  
import java.util.List;  
  
public class ListaClientes {  
    private List<Cliente> lista = new ArrayList<Cliente>();  
  
    public ListaClientes(){  
    }  
  
    public void add(Cliente cliente) {  
        lista.add(cliente);  
    }  
  
    public List<Cliente> getListaClientes() {  
        return lista;  
    }  
}
```

```

import java.io.EOFException;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;

import com.thoughtworks.xstream.XStream;

public class EscribirClientes {
    public static void main(String[] args)
        throws IOException, ClassNotFoundException {
        File fichero = new
File("C:\\AccesoDatos\\UD1\\Ejercicios\\FichCliente.dat");
        FileInputStream filein = new
            FileInputStream(fichero); //flujo de entrada
        //conecta el flujo de bytes al flujo de datos
        ObjectInputStream dataIS = new ObjectInputStream(filein);
        System.out.println
            ("Comienza el proceso de creación del fichero a XML ...");

        //Creamos un objeto Lista de Clientes
        ListaClientes listacli = new ListaClientes();

        try {
            while (true) { //lectura del fichero
                //leer una Persona
                Cliente cliente= (Cliente) dataIS.readObject();
                listacli.add(cliente); //añadir cliente a la lista
            }
        } catch (EOFException eo) {}
        dataIS.close(); //cerrar stream de entrada

        try {
            XStream xstream = new XStream();
            //cambiar de nombre a las etiquetas XML
            xstream.alias("ListaClientesMunicipio",
                ListaClientes.class);
            xstream.alias("DatosCliente", Cliente.class);

            //xstream.aliasField("Nombre cliente", Cliente.class, "nombre");
            //xstream.aliasField("Edad cliente", Cliente.class, "edad");

            //quitar etiqueta lista (atributo de la clase ListaClientes)
            xstream.addImplicitCollection
                (ListaClientes.class, "lista");
            //Insertar los objetos en el XML
            xstream.toXML(listacli, new
FileOutputStream("C:\\AccesoDatos\\UD1\\Ejercicios\\Clientes.xml"));
            System.out.println("Creado fichero XML....");

        } catch (Exception e)
            {e.printStackTrace();}
    }
}

```

```
} // fin main  
  
} //fin EscribirPersonas
```

El proceso para realizar la lectura del fichero XML generado es el siguiente:

```
import java.io.*;  
import java.util.ArrayList;  
import java.util.Iterator;  
import java.util.List;  
import java.util.ListIterator;  
  
import com.thoughtworks.xstream.XStream;  
  
public class LeerClientes {  
    public static void main(String[] args) throws IOException {  
  
        XStream xstream = new XStream();  
  
        xstream.alias("ListaClientes", ListaClientes.class);  
        xstream.alias("DatosCliente", Cliente.class);  
        xstream.addImplicitCollection(ListaClientes.class, "lista");  
  
        //importante por razones de seguridad si no se pone la siguiente línea  
        //dará error de ForbiddenClassException  
        xstream.addPermission(AnyTypePermission.ANY);  
  
        ListaClientes listadoTodas = (ListaClientes)  
            xstream.fromXML(new  
FileInputStream("C:\\AccesoDatos\\UD1\\Ejercicios\\Personas.xml"));  
  
        System.out.println("Numero de Personas: " +  
            listadoTodas.getListaClientes().size());  
  
        List<Cliente> listaClientes = new ArrayList<Cliente>();  
        listaClientes = listadoTodas.getListaClientes();  
  
        ListIterator<Cliente> iterador = listaClientes.listIterator();  
        while( iterador.hasNext() ) {  
            Cliente cliente = (Cliente) iterador.next();  
            System.out.println("Nombre: "+ cliente.getNombre());  
            System.out.println("Edad: "+cliente.getEdad());  
        }  
        System.out.println("Fin de listado .....");  
    } //fin main  
  
} //fin LeerPersonas
```

7.4 Conversión de ficheros XML a otro formato

Las hojas de estilos XSL transforman el contenido de los documentos XML (XSLT), y les aplican formato (objetos de formato) de cara a su visualización en un navegador. También es posible utilizar XSLT en el servidor para transformar XML en HTML y poder visualizar así, el contenido del documento en navegadores que no cuenten con soporte para XML. En el siguiente ejemplo se ve cómo a partir de un fichero XML que contiene datos y otro XSL que contiene la presentación de esos datos, se puede generar un fichero HTML usando el lenguaje Java.

Contenido fichero documento **empleados.xml**

```
<?xml version="1.0"?>
<listadeempleados>
  <empleado>
    <nombre>Antonio</nombre>
    <departamento>10</departamento>
  </empleado>
  <empleado>
    <nombre>Laura</nombre>
    <departamento>20</departamento>
  </empleado>
</listadeempleados>
```

Contenido fichero clientesPlantilla.xml:

```
<?xml version="1.0" encoding="iso-8859-1" ?>

<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

  <xsl:template match="/">

    <html><xsl:apply-templates /></html>

  </xsl:template>

  <xsl:template match='listadeempleados'>

    <head><title>LISTADO DE EMPLEADOS</title></head>

    <body>

      <h1>LISTA DE EMPLEADOS</h1>

      <table border='1'>

        <tr>

          <th>Nombre</th>

          <th>Departamento</th>

        </tr>

        <xsl:apply-templates select='empleado' />

      </table>

    </body>

  </xsl:template>

  <xsl:template match='empleado'>

    <tr><xsl:apply-templates /></tr>

  </xsl:template>

  <xsl:template match='nombre|departamento'>
```

```

        <td><xsl:apply-templates /></td>

    </xsl:template>

</xsl:stylesheet>

```

Para realizar la transformación se necesita obtener un objeto **Transformer** que se obtiene creando una instancia de **TransformerFactory** y aplicando el método `newTransformer(Source source)` a la fuente XSL, y aplicar la transformación del fichero de datos XML, o lo que es lo mismo, aplicar la hoja de estilos XSL al fichero XML:

```

Transformer transformer =
TransformerFactory.newInstance().newTransformer(estilos);

```

```

transformer.transform(datos, resultado);

```

El código completo resultaría así:

```

import java.io.File;
import java.io.FileOutputStream;
import java.io.IOException;

import javax.xml.transform.Source;
import javax.xml.transform.Transformer;
import javax.xml.transform.TransformerFactory;
import javax.xml.transform.stream.StreamResult;
import javax.xml.transform.stream.StreamSource;

public class Convertidor {

    public static void main(String[] args) throws IOException
    {
        // TODO Auto-generated method stub
        String hojaEstilo =
"C:\\AccesoDatos\\UD1\\Ejercicios\\clientesPlantilla.xsl";
        String datosClientes =
"C:\\AccesoDatos\\UD1\\Ejercicios\\empleados.xml";
        File paginaHTML = new
File("C:\\AccesoDatos\\UD1\\Ejercicios\\index.html");

        //Crear fichero HTML
        FileOutputStream os = new
FileOutputStream(paginaHTML);
        Source estilos = new StreamSource(hojaEstilo);

        //Fuente XSL
        Source datos = new StreamSource(datosClientes);

        //Fuente XML

        //Resultado de la transformación
        javax.xml.transform.Result result = new
StreamResult(os);

```

```

        try {
            Transformer transformer =
TransformerFactory.newInstance().newTransformer(estilos);
            transformer.transform(datos, result);
        } catch (Exception e) {
            // TODO: handle exception
            System.err.println("Error: "+e);
        }
        finally {
            os.close();
        }
    }
}

```

El fichero HTML resultado se muestra en la figura 1.3



Figura 1.3 Página HTML resultado (Java + XSL)

VIII. EXCEPCIONES: DETECCIÓN Y TRATAMIENTO

Una excepción en Java es una clase que representa la información recogida sobre una situación excepcional. Una situación excepcional no tiene por qué ser un error, aunque todos los errores son (o deberían ser) situaciones excepcionales. Por ejemplo, una situación excepcional sería la falta de fondos en una cuenta de un banco, pero eso no es un error sino una situación excepcional que depende únicamente del tipo de aplicación que se está desarrollando. Por otro lado, acceder a una posición negativa de una matriz es una excepción y un error que se debe corregir. Por lo tanto, las excepciones no tienen por qué ser errores. En Java existen excepciones de tiempo de ejecución y excepciones del programa. Las excepciones en Java deben capturarse y tratarse obligatoriamente salvo en el caso de las excepciones de tiempo de ejecución, que se consideran errores de programación que hay que solucionar.